BLOCK Data Engineering - Unit on Feature Selection / Part 2

Myra Spiliopoulou [1]

[1] Faculty of Computer Science, Otto-von-Guericke University Magdeburg

1 Feature Selection Process

2 Filters & Wrappers

3 Removing to Explain

4 Closing

Feature Selection: What and Why

Definition of FSel [Section 7.1]

'*Feature selection* is a process that chooses the optimal subset of features according to a certain criterion.'

Feature Selection: What and Why

Definition of FSel [Section 7.1]

'*Feature selection* is a process that chooses the optimal subset of features according to a certain criterion.'

Optimality / Goodness ⇒ Purpose of the feature selection process [a]

[a] From Section 4.1 and from citation [48]:Sayes Y., Inza I, Larranaga P (2007) A review of feature selection techniques in bioinformatics. *Bioinformatics* 23(19), 2507-2517

- ⋆ discard redundant features
- ⋆ reduce the cost of data acquisition

Feature Selection: What and Why

Definition of FSel [Section 7.1]

'*Feature selection* is a process that chooses the optimal subset of features according to a certain criterion.'

Optimality / Goodness ⇒ Purpose of the feature selection process [a]

[a] From Section 4.1 and from citation [48]:Sayes Y., Inza I, Larranaga P (2007) A review of feature selection techniques in bioinformatics. *Bioinformatics* 23(19), 2507-2517

- ⋆ discard redundant features
- ⋆ reduce the cost of data acquisition

and when applying FSel in preparation of an already known learning task:

- ○ discard irrelevant features, and thus also irrelevant data
- ○ increase the accuracy/quality of the learned models
- ○ reduce the complexity of the model / of the model description
- ○ improve efficiency of the learning process, e.g. by reducing storage requirements and computational costs

The Quest for the Good Features (Section 7.2.1.1, with modifications)

Given a feature space F and a goodness measure U: Build the subset of the good features S from scratch or rather by discarding features from F ?

The Quest for the Good Features (Section 7.2.1.1, with modifications)

Given a feature space F and a goodness measure U: Build the subset of the good features S from scratch or rather by discarding features from F ?

Sequential forward and backward feature set generation algorithms

SFG	Sequential forward feature set generation
function	$SFG(F,U)$
initialize	$S = \emptyset$
repeat	
	$f = FindNextBest(F,U)$
	$S = S \cup \{f\}$
	$F = F \setminus \{f\}$
until	S satisfies U or $F = \emptyset$
return	S
end function	

SBG	Sequential backward feature set generation
function	$SBG(F,U)$
initialize	$S = \emptyset$
repeat	
	$f = FindNextWorst(F,U)$
	$S = S \cup \{f\}$
	$F = F \setminus \{f\}$
until	F does not satisfy U or $F = \emptyset$
return	$F \cup \{f\}$ `// add the last feature again`
end function	

The Quest for the Good Features (Section 7.2.1.1, with modifications)

Given a feature space F and a goodness measure U: Build the subset of the good features S from scratch or rather by discarding features from F ?

Sequential forward and backward feature set generation algorithms

SFG	Sequential forward feature set generation
function	$SFG(F,U)$
initialize	$S = \emptyset$
repeat	
	$f = FindNextBest(F,U)$
	$S = S \cup \{f\}$
	$F = F \setminus \{f\}$
until	S satisfies U or $F = \emptyset$
return	S
end function	

SBG	Sequential backward feature set generation
function	$SBG(F,U)$
initialize	$S = \emptyset$
repeat	
	$f = FindNextWorst(F,U)$
	$S = S \cup \{f\}$
	$F = F \setminus \{f\}$
until	F does not satisfy U or $F = \emptyset$
return	$F \cup \{f\}$ `// add the last feature again`
end function	

where the $FindNextBest()$ and $FindNextWorst()$ functions operate on the feature set as it is modified in each iteration (rather than the original one). Hence, each newly selected feature f is best/worst given the previous selections.

The Quest cntd

(Algorithms 3 & 4, with modifications)

Combining SFG and SBG into: Bidirectional feature set generation algorithm

BG	Bidirectional feature set generation
function	$BG(F,U)$
initialize	$S_g = \emptyset$; $S_b = \emptyset$; $F_g = F$; $F_b = F$
repeat	
	$f_g = FindNextBest(F_g, U)$; $S_g = S_g \cup \{f_g\}$; $F_g = F_g \setminus \{f_g\}$
	$f_b = FindNextWorst(F_b, U)$; $S_b = S_b \cup \{f_b\}$; $F_b = F_b \setminus \{f_b\}$
until	$(a) S_g$ satisfies U or $F_g = \emptyset$ or (b) F_b does not satisfy U or $F_b = \emptyset$
if	(a) holds *then return* S_g *else return* $F_b \cup \{f_b\}$ *endif*
end function	

The Quest cntd

(Algorithms 3 & 4, with modifications)

Combining SFG and SBG into: Bidirectional feature set generation algorithm

BG	Bidirectional feature set generation
function	$BG(F, U)$
initialize	$S_g = \emptyset$; $S_b = \emptyset$; $F_g = F$; $F_b = F$
repeat	
	$f_g = FindNextBest(F_g, U)$; $S_g = S_g \cup \{f_g\}$; $F_g = F_g \setminus \{f_g\}$
	$f_b = FindNextWorst(F_b, U)$; $S_b = S_b \cup \{f_b\}$; $F_b = F_b \setminus \{f_b\}$
until	$(a) S_g$ satisfies U or $F_g = \emptyset$ or (b) F_b does not satisfy U or $F_b = \emptyset$
if	(a) holds *then return* S_g *else return* $F_b \cup \{f_b\}$ *endif*
end function	

Generating *subsets* of F at random:

RG	Random feature set generation
function	$RG(F, U)$
initialize	$S = S_{best} \emptyset$; $c_{best} = \|F\|$
repeat	
	$S = \mathit{RandGen}(F)$ `// pick features from F at random`
	if $\|S\| \leq c_{best}$ and S satisfies U *then* $S_{best} = S$; $c_{best} = \|S\|$ *endif*
until	some stopping criterion is satisfied
return	S_{best}
end function	

How long to search? (Section 7.2.1.2, with modifications)

- *Exhaustive search:*
  Generate all non-empty subsets of F, i.e. $\mathscr{P}(F) \setminus \{\emptyset\}$, and choose the best one
  - ok if F is small
  - nok if F is large $\Rightarrow$ set a threshold m as the minimum number of features to be selected or removed
- *Heuristic search:*
  Traverse the search space of solutions by adhering to some heuristic, which also incorporates a termination criterion
- *Non-deterministic search:*
  Generate subsets of F at random (cf. Algorithm RG) without an explicit termination criterion $\Rightarrow$ at each time point, RG() returns the best subset found thus far [1]

[1] In 'stream mining', best effort algorithms of this kind are called *anytime algorithms*.

How long to search? (Section 7.2.1.2, with modifications)

- *Exhaustive search:*
  Generate all non-empty subsets of F, i.e. $\mathscr{P}(F) \setminus \{\emptyset\}$, and choose the best one
  - ok if F is small
  - nok if F is large $\Rightarrow$ set a threshold m as the minimum number of features to be selected or removed
- *Heuristic search:*
  Traverse the search space of solutions by adhering to some heuristic, which also incorporates a termination criterion
- *Non-deterministic search:*
  Generate subsets of F at random (cf. Algorithm RG) without an explicit termination criterion $\Rightarrow$ at each time point, RG() returns the best subset found thus far [1]

To which category does each of SFG, SBG and BG belong?

[1] In 'stream mining', best effort algorithms of this kind are called *anytime algorithms*.

Moving on to featureset selection EXAMPLE

From Table 1 in [Schleicher et al., 2024]:

Pos	Feature-set	# Items	Outcome of interest
1	SOZK	25	-
2	TQ	52	YES
3	PSQ	30	YES
4	SF8	8	–
5	SWOP	9	–
6	TLQ	8	–
7	BSF	30	–
8	BI	56	–
9	ADSL	20	YES
		238	YES: 102

Moving on to featureset selection EXAMPLE

From Table 1 in [Schleicher et al., 2024]:

Pos	Feature-set	# Items	Outcome of interest
1	SOZK	25	-
2	TQ	52	YES
3	PSQ	30	YES
4	SF8	8	–
5	SWOP	9	–
6	TLQ	8	–
7	BSF	30	–
8	BI	56	–
9	ADSL	20	YES
		238	YES: 102

Constraints on Featureset selection

- Featuresets with a YES cannot be skipped.
- Each featureset contains features that correlate to features of another featureset, BUT
- a featureset cannot be broken; it is selected whole-or-nothing.

Moving on to featureset selection EXAMPLE

From Table 1 in [Schleicher et al., 2024]:

Pos	Featureset	# Items	Outcome of interest
1	SOZK	25	-
2	TQ	52	YES
3	PSQ	30	YES
4	SF8	8	–
5	SWOP	9	–
6	TLQ	8	–
7	BSF	30	–
8	BI	56	–
9	ADSL	20	YES
		238	YES: 102

Constraints on Featureset selection

- Featuresets with a YES cannot be skipped.
- Each featureset contains features that correlate to features of another featureset, BUT
- a featureset cannot be broken; it is selected whole-or-nothing.

Solutions

- Apply SFG / SBG on featuresets by removing one featureset at a time
- Identify featuresets that predict others [Schleicher et al., 2024]
- Rank featuresets on
  - cost [Schleicher et al., 2024]
  - incurred missingness [Puga et al., 2023]

Thus far: We have seen what feature selection means, and what reasons are there to perform a feature selection process.

We have seen some ways of building a 'good' subset of the original set of features, assuming a criterion of 'goodness', such as redundancy (negative criterion) or good predictive power towards a target (positive criterion)

Thus far: We have seen what feature selection means, and what reasons are there to perform a feature selection process.
We have seen some ways of building a 'good' subset of the original set of features, assuming a criterion of 'goodness', such as redundancy (negative criterion) or good predictive power towards a target (positive criterion)

Your turn: You must be able to explain the four feature set construction algorithms, name their termination criteria (if any), and say whether each of them is exhaustive, heuristic or non-deterministic – and explain why.

Thus far: We have seen what feature selection means, and what reasons are there to perform a feature selection process.
We have seen some ways of building a 'good' subset of the original set of features, assuming a criterion of 'goodness', such as redundancy (negative criterion) or good predictive power towards a target (positive criterion)

Your turn: You must be able to explain the four feature set construction algorithms, name their termination criteria (if any), and say whether each of them is exhaustive, heuristic or non-deterministic – and explain why.

What comes next: Filters & Wrappers

1 Feature Selection Process

2 Filters & Wrappers

3 Removing to Explain

4 Closing

What is missing from the feature selection workflow?

We already have

- $\surd$ functions that help us decide whether a feature is good to keep or rather to discard
- $\surd$ procedures to traverse the search space, i.e. all combinations of features, and create feature subsets
- $\surd$ stopping criteria

What is missing from the feature selection workflow?

We already have

- $\surd$ functions that help us decide whether a feature is good to keep or rather to discard
- $\surd$ procedures to traverse the search space, i.e. all combinations of features, and create feature subsets
- $\surd$ stopping criteria

but how good is our constructed subset of features ? for a learning algorithm ?

What is missing from the feature selection workflow?

We already have

- $\surd$ functions that help us decide whether a feature is good to keep or rather to discard
- $\surd$ procedures to traverse the search space, i.e. all combinations of features, and create feature subsets
- $\surd$ stopping criteria

but how good is our constructed subset of features ? for a learning algorithm ?

Training and testing

Given a sample of the data D that is representative of the population under study:

- We split D into a training set D_{train} and a test set D_{test} so that $D = D_{train} \cup D_{test}$ and $D_{train} \cap D_{test} = \emptyset$.
- We use D_{train} to train the learning algorithm and induce a model M
- We use D_{test} to test the behaviour of M on previously unseen data

What is missing from the feature selection workflow?

We already have

- $\surd$ functions that help us decide whether a feature is good to keep or rather to discard
- $\surd$ procedures to traverse the search space, i.e. all combinations of features, and create feature subsets
- $\surd$ stopping criteria

but how good is our constructed subset of features ? for a learning algorithm ?

$\swarrow$

Training and testing

Given a sample of the data D that is representative of the population under study:

- We split D into a training set D_{train} and a test set D_{test} so that $D = D_{train} \cup D_{test}$ and $D_{train} \cap D_{test} = \emptyset$.
- We use D_{train} to train the learning algorithm and induce a model M
- We use D_{test} to test the behaviour of M on previously unseen data

$\longrightarrow$

? How to test the impact of the feature selection process on the model M

? And WHEN?

Filters (Section 7.2.3.1) & Wrappers (Section 7.2.3.2)

Filters

filter the good from the no-good features *before learning*.

Stage 1: Feature subset selection with a goodness criterion and a feature set generation algorithm – it delivers its best subset of features

Stage 2: The learning algorithm uses the data of this subset of features only, first for training, then for testing

Subcategory **Rankers**: Stage 1 delivers all features, each one ranked on goodness; at stage 2, the algorithm applies a threshold to pick the top-ranked features

Filters (Section 7.2.3.1) & Wrappers (Section 7.2.3.2)

Filters

filter the good from the no-good features *before learning*.

Stage 1: Feature subset selection with a goodness criterion and a feature set generation algorithm – it delivers its best subset of features

Stage 2: The learning algorithm uses the data of this subset of features only, first for training, then for testing

Subcategory **Rankers**: Stage 1 delivers all features, each one ranked on goodness; at stage 2, the algorithm applies a threshold to pick the top-ranked features

Wrappers

engage a classifier to decide whether a feature should be kept or discarded, depending on its impact on classification quality.

Stage 1: Feature subset selection, where the goodness function is in the blackbox of the classification algorithm – it delivers the best subset of features, from the viewpoint of the classifier

Stage 2: as for Filters, but

- ! a model must be induced on the best subset of features
- ! and tested on data that have not been seen before

Thus far: We have completed the workflow of feature selection, and seen two forms of implementing a workflow - through a filter or a wrapper

Thus far: We have completed the workflow of feature selection, and seen two forms of implementing a workflow - through a filter or a wrapper

Your turn: You must be able to explain a filter, a wrapper, and the differences between them.

Thus far: We have completed the workflow of feature selection, and seen two forms of implementing a workflow - through a filter or a wrapper

Your turn: You must be able to explain a filter, a wrapper, and the differences between them.

What comes next: A feature's contribution to model quality is used in the context of explaining (black-box) models, so we look a bit on how to find features that are good in explaining a model's behavior.

1. Feature Selection Process
2. Filters & Wrappers
3. **Removing to Explain**
4. Closing

Feature Goodness from the Explanation Perspective

Core idea: to find how well a feature explains a model's behavior, remove the feature and describe the effected change in the model

Removing to explain [Covert et al., 2021, Figure 1]

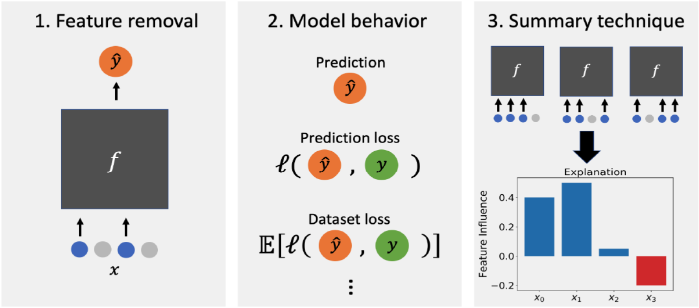

Figure 1: A unified framework for *removal-based explanations*. Each method is determined by three choices: how it removes features, what model behavior it analyzes, and how it summarizes feature influence.

. . . and 26 removal-based explainers, placed in the unified framework

CHOICE 1. How to withhold features from the model? [Covert et al., 2021]

- ⋆ Build one model for each subset of features
  APPROACH USED e.g. BY 'Shapley Net Effects': *Stan Lipovetsky and Michael Conklin. Analysis of regression in game theory approach. Applied Stochastic Models in Business and Industry, 17(4):319–330, 2001.*

CHOICE 1. How to withhold features from the model? [Covert et al., 2021]

- ⋆ Build one model for each subset of features
  APPROACH USED e.g. BY 'Shapley Net Effects': *Stan Lipovetsky and Michael Conklin. Analysis of regression in game theory approach. Applied Stochastic Models in Business and Industry, 17(4):319–330, 2001.*
- ⋆ Set the values to zero
- ⋆ Set the values to user-defined defaults – to gray pixels, to the mean, . . .

Example: removing a caterpillar's nose


- ⋆ Set the values to zero *and* train a model to recognize these zeroes as missing values
  APPROACH USED e.g. BY 'INVASE': *Jinsung Yoon, James Jordon, and Mihaela van der Schaar. INVASE: Instance-wise variable selection using neural networks. In Int. Conf. on Learning Representations, 2018.*
- ⋆ . . .

CHOICE 2. How to quantify change in model behavior? [Covert et al., 2021]

- ⋆ Change in prediction (quality) for an individual instance
  'Occlusion, RISE, PredDiff, MP, EP, MM, FIDO-CA, MIR, LIME, SHAP (including KernelSHAP and TreeSHAP), IME and QII . . . examine how holding out **different** features makes an individual prediction either higher or lower', cf. [Covert et al., 2021, page 15, 1st bulletpoint]
- ⋆ 'Dataset loss', ie loss in prediction quality , whereby the loss may be negative, over the whole dataset
  'Shapley Net Effects' is among the methods that use this approach.
- ⋆ . . .
- ⋆ Prediction loss / Dataset loss *with respect to output*, i.e. comparing the prediction (quality) after feature removal to that of the full model

CHOICE 3. How to summarize a feature's influence? [Covert et al., 2021]

Core idea: Assuming a 'set function' that represent a model's dependence on different features, we want to calculate each feature's 'attribution' to this function.

1) Calculate the impact of removing an individual feature [recall SBG]
2) Calculate the impact of including an individual feature [recall SFG]
3) Mean of impact when including an individual feature, computed over many subsamples
4) Solve the optimization problem of building a (minimal) subset of features that suffice to achieve a high value for the set function
5) Solve the optimization problem of building a (maximal) subset of features that can be removed without affecting the set function
6) Compute the Shapley value of each feature cf [Covert et al., 2021, S. 7]
7) ...

Thus far: We have seen a unified framework that summarizes all explainers into a three-stage process. We have looked at the stages and found 'goodness' in the 2nd and 3rd stage.

Thus far: We have seen a unified framework that summarizes all explainers into a three-stage process. We have looked at the stages and found 'goodness' in the 2nd and 3rd stage.

Your turn: You should be able to explain the three stages/choices, and to answer questions like

- Does the unified framework of [Covert et al., 2021] correspond to a filter or to a wrapper? Or neither? Explain.
- If two features are correlated, which of the approaches under Choice 3 will lead to the elimination of one of the features? Explain.

1. Feature Selection Process
2. Filters & Wrappers
3. Removing to Explain
4. **Closing**

Summary and Outlook

We have seen

a workflow for the feature selection process:

- $\surd$ Ways of building up a feature subset by adding or discarding features
- $\surd$ Functions that quantify the goodness of features
- $\surd$ Two ways of connecting feature selection to model learning: filter the feature space before learning, or wrap the learning algorithm into the feature selection workflow
- $\surd$ A link between feature selection and feature removal for model explanation

Instruments that we skipped:

- Methods for feature *construction* in a projected space, e.g. 'Principal Component Analysis'
- Elaborate descriptions of a feature's contribution towards predicting a target – cf [Covert et al., 2021, Sections 6, 7]

Thank you very much!

Questions?

Bibliography I

- Covert, I., Lundberg, S., and Lee, S.-I. (2021). Explaining by removing: A unified framework for model explanation. *Journal of Machine Learning Research*, 22(209):1–90.
- Puga, C., Niemann, U., Schlee, W., and Spiliopoulou, M. (2023). A cost-based multi-layer network approach for the discovery of patient phenotypes. *International Journal of Data Science and Analytics (JDSA)*.
- Schleicher, M., Brüggemann, P., Böcking, B., Niemann, U., Mazurek, B., and Spiliopoulou, M. (2024). Parsimonious predictors for medical decision support: Minimizing the set of questionnaires used for tinnitus outcome prediction. *Expert Systems with Applications*, 239:122336.